Spring Security - Database Authentication 1
1. Spring Security - Database Authentication Implementation 1
2. Implementation of HTTP Basic authentication 1
3. InMemory Authentication 2
4. Database authentication implementation process - 5
5. References - 14

Spring Security - Database Authentication

1. Spring Security - Database Authentication Implementation

Step 1 - Add dependency

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Step 2 - Run the application
Auto generated passwords will be there which can be used to do the form based authentication.

User defined username and password with roles:

```
#spring security
spring.security.user.name=sourabh
spring.security.user.password=sourabh
spring.security.user.roles=ADMIN
```

2. Implementation of HTTP Basic authentication

Step 1 - create a package name config

Step 2 - create a class name SecurityConfig and annotate it with @Configuration
This annotation makes this class a configuration for security
Step 3 - create a Security filter chain method and annotate it with @Bean

```
package com.springboot.blog.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.builders.HttpSecuri
ty;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

   //http basic authentication logic using SecurityFilterChain
   @Bean
   SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       http
               .csrf().disable()
               .authorizeHttpRequests((authorize)->
authorize.anyRequest().authenticated())
               .httpBasic(Customizer.withDefaults());
       return http.build();
   }
}
```

Step 4 - run and check in the browser and also in the Postman
Username and Password are encrypted using base64 technique

3. InMemory Authentication

Step 1 - update SecurityConfig class

```
package com.glo.app.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.En
ableMethodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecuri
ty;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import
org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

   //PasswordEncoder using base64 format
   @Bean
   PasswordEncoder passwordEncoder(){
       return new BCryptPasswordEncoder();
   }

   //http basic authentication logic using SecurityFilterChain
   @Bean
   SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       http
```

```
                .csrf().disable()
                .authorizeHttpRequests(

(authorize)->authorize.requestMatchers(HttpMethod.GET,"/api/employees/
**").permitAll()
                                    .anyRequest().authenticated()
                ).httpBasic(Customizer.withDefaults());
        return http.build();
    }

    //In memory authentication using UserDetailsService
    @Bean
    UserDetailsService userDetailsService(){
        UserDetails sourabh =
User.builder().username("Sourabh").password(passwordEncoder().encode("
Sourabh")).roles("USER").build();
        UserDetails admin =
User.builder().username("Admin").password(passwordEncoder().encode("Ad
min")).roles("ADMIN").build();

        return new InMemoryUserDetailsManager(sourabh,admin);
    }

}
```

Step 2 - Add the @PreAuthorize annotation in the Controller class

```
package com.glo.app.controller;

import com.glo.app.payload.EmployeeDto;
import com.glo.app.payload.EmployeeResponse;
import com.glo.app.service.EmployeeService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
```

```
        this.employeeService = employeeService;
    }

    //Spring boot REST api for createEmployee
    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<EmployeeDto> createEmployee(
            @Valid @RequestBody EmployeeDto employeeDto
    ){
        return new
ResponseEntity(employeeService.createEmployee(employeeDto),
HttpStatus.CREATED);
    }

    //Spring boot REST api for getEmployees
    @GetMapping
    public EmployeeResponse getEmployees(
            @RequestParam(value = "pageNo",defaultValue = "0",required
= false) int pageNo,
            @RequestParam(value = "pageSize",defaultValue =
"10",required = false) int pageSize,
            @RequestParam(value = "sortBy",defaultValue = "id",required
= false) String sortBy,
            @RequestParam(value = "sortDir",defaultValue =
"asc",required = false) String sortDir
    ){
        return
employeeService.getEmployees(pageNo,pageSize,sortBy,sortDir);
    }

    //Spring boot REST api for getEmployeeById
    @GetMapping("/{id}")
    public EmployeeDto getEmployeeById(@PathVariable("id") long id){
        return employeeService.getEmployeeById(id);
    }

    //Spring boot REST api for updateEmployee
    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public EmployeeDto updateEmployee(
            @PathVariable("id") long id,
            @RequestBody EmployeeDto employeeDto
    ){
        return employeeService.updateEmployee(id,employeeDto);
```

```
    }

    //Spring boot REST api for deleteEmployee
    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteEmployee(@PathVariable("id") long id){
        employeeService.deleteEmployee(id);
        return "Employee deleted successfully";
    }
}
```

Step 3 - Test using PostMan

Create AccessDeniedException in Global Exception Handler class

```
//handle the specific exception
//access denied  exception
@ExceptionHandler(AccessDeniedException.class)
public ResponseEntity<ErrorDetails> handleAccessDeniedException(
        AccessDeniedException accessDeniedException,
        WebRequest webRequest
){
    ErrorDetails errorDetails = new ErrorDetails(new
Date(),accessDeniedException.getMessage(),
webRequest.getDescription(false));
    return new ResponseEntity(errorDetails, HttpStatus.UNAUTHORIZED);
}
```

4. Database authentication implementation process -

User and Role with many to many mapping for authentication and authorization

Step 1 - create Role and User entity
User entity with many to many relationship with Role entity

```
package com.glo.app.payload;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.util.Set;
```

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private long id;
   private String name;
   @Column(nullable = false,unique = true)
   private String email;
   @Column(nullable = false,unique = true)
   private String username;
   @Column(nullable = false)
   private String password;
   @ManyToMany(fetch = FetchType.EAGER,cascade = CascadeType.ALL)
   @JoinTable(name = "user_roles",
           joinColumns = @JoinColumn(name =
"user_id",referencedColumnName = "id"),
           inverseJoinColumns = @JoinColumn(name =
"role_id",referencedColumnName = "id"))
   private Set<Role> role;
}
```

Role Entity

```
package com.glo.app.payload;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Setter
@Getter
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "roles")
public class Role {
   @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column(nullable = false,unique = true)
    private String name;
}
```

Step 2- run the application and check the MySQL database

Step 3 - Create the User and Role Repository

UserRepository

```
package com.glo.app.repository;

import com.glo.app.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User,Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
    Optional<User> findByUsernameOrEmail(String username,String email);
    Boolean existsByUsername(String username);
    Boolean existsByEmail(String email);
}
```

RoleRepository

```
package com.glo.app.repository;

import com.glo.app.entity.Role;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface RoleRepository extends JpaRepository<Role,Long> {
    Optional<Role> findByName(String name);
}
```

Step 4 - Create the security package and create the implementation to interact with database

CustomUserDetailsService class implements UserDetailsService

```
package com.glo.app.security;

import com.glo.app.entity.User;
```

```
import com.glo.app.repository.UserRepository;
import org.springframework.security.core.GrantedAuthority;
import
org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundExceptio
n;
import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.stream.Collectors;

@Service
public class CustomUserDetailsService implements UserDetailsService {
   private UserRepository userRepository;

   //Constructor based dependency injection
   public CustomUserDetailsService(UserRepository userRepository) {
       this.userRepository = userRepository;
   }

   @Override
   public UserDetails loadUserByUsername(String usernameOrEmail)
throws UsernameNotFoundException {
       //database interaction
       User user =
userRepository.findByUsernameOrEmail(usernameOrEmail,
usernameOrEmail).orElseThrow(
               () -> new UsernameNotFoundException("User doesn't
exists by username or email: " + usernameOrEmail)
       );
       //converting the roles to authorities
       Set<GrantedAuthority> authorities =
               user.getRole().stream().map(r->new
SimpleGrantedAuthority(r.getName())).collect(Collectors.toSet());

       //returning the user object of security framework with email,
password and roles in the form of authorities
       return new
org.springframework.security.core.userdetails.User(user.getEmail(),use
r.getPassword(),authorities);
```

```
    }
}
```

LoginDto
Step 1 - Create LoginDto

```
package com.glo.app.payload;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Setter
@Getter
@AllArgsConstructor
@NoArgsConstructor
public class LoginDto {
    private String usernameOrEmail;
    private String password;
}
```

Step 2 - Create the service layer for AuthService and AuthServiceImpl

AuthService

```
package com.glo.app.service;

import com.glo.app.payload.LoginDto;

public interface AuthService {
    String login(LoginDto loginDto);

}
```

AuthServiceImpl

```
package com.glo.app.service.impl;

import com.glo.app.payload.LoginDto;
import com.glo.app.repository.UserRepository;
import com.glo.app.service.AuthService;
```

```
import
org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthentica
tionToken;
import org.springframework.security.core.Authentication;
import
org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;

@Service
public class AuthServiceImpl implements AuthService {
   //to create the token for authentication process
   private AuthenticationManager authenticationManager;

   //Constructor based dependency injection
   public AuthServiceImpl(UserRepository userRepository,
AuthenticationManager authenticationManager) {
       this.authenticationManager = authenticationManager;
   }

   @Override
   public String login(LoginDto loginDto) {
       Authentication authenticate =
               authenticationManager.
                       authenticate(new
UsernamePasswordAuthenticationToken(loginDto.getUsernameOrEmail(),
loginDto.getPassword()));


SecurityContextHolder.getContext().setAuthentication(authenticate);
       return "User logged in successfully!...";
   }
}
```

Step 3 - Create AuthController

```
package com.glo.app.controller;

import com.glo.app.payload.LoginDto;
import com.glo.app.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private AuthService authService;
    //Constructor based dependency injection
    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    //Spring boot REST api for login
    @PostMapping(value = {"/sign-in","/login"})
    public ResponseEntity<String> login(
            @RequestBody LoginDto loginDto
    ){
        return ResponseEntity.ok(authService.login(loginDto));
    }
}
```

RegisterDto

Step 1 - create RegisterDto

```
package com.glo.app.payload;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Setter
@Getter
@AllArgsConstructor
@NoArgsConstructor
//DTO class to register the new user in the system
public class RegisterDto {
    private long id;
    private String name;
    private String email;
    private String username;
    private String password;
}
```

Step 2 - update AuthService and AuthServiceImpl

AuthService

```
package com.glo.app.service;

import com.glo.app.payload.LoginDto;
import com.glo.app.payload.RegisterDto;

public interface AuthService {
   String login(LoginDto loginDto);
   String register(RegisterDto registerDto);
}
```

AuthServiceImpl

```
package com.glo.app.service.impl;

import com.glo.app.entity.Role;
import com.glo.app.entity.User;
import com.glo.app.exception.EmployeeApiException;
import com.glo.app.payload.LoginDto;
import com.glo.app.payload.RegisterDto;
import com.glo.app.repository.RoleRepository;
import com.glo.app.repository.UserRepository;
import com.glo.app.service.AuthService;
import org.springframework.http.HttpStatus;
import
org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthentica
tionToken;
import org.springframework.security.core.Authentication;
import
org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.HashSet;
import java.util.Set;

@Service
public class AuthServiceImpl implements AuthService {
   //to create the token for authentication process
   private AuthenticationManager authenticationManager;
   private UserRepository userRepository;
   private RoleRepository roleRepository;
```

```
    private PasswordEncoder passwordEncoder;

    //Constructor based dependency injection
    public AuthServiceImpl(AuthenticationManager authenticationManager,
                           UserRepository userRepository,
                           RoleRepository roleRepository,
                           PasswordEncoder passwordEncoder) {
        this.authenticationManager = authenticationManager;
        this.userRepository = userRepository;
        this.roleRepository = roleRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public String login(LoginDto loginDto) {
        Authentication authenticate =
                authenticationManager.
                        authenticate(new
UsernamePasswordAuthenticationToken(loginDto.getUsernameOrEmail(),
loginDto.getPassword()));


SecurityContextHolder.getContext().setAuthentication(authenticate);
        return "User logged in successfully!...";
    }

    @Override
    public String register(RegisterDto registerDto) {
        //check username
        if(userRepository.existsByUsername(registerDto.getUsername()))
            throw new
EmployeeApiException(HttpStatus.BAD_REQUEST,"Username already
exists");

        //check email
        if(userRepository.existsByEmail(registerDto.getEmail()))
            throw new
EmployeeApiException(HttpStatus.BAD_REQUEST,"Email already exists");

        //set the user object
        User user = new User();
        user.setName(registerDto.getName());
        user.setEmail(registerDto.getEmail());
        user.setUsername(registerDto.getUsername());
```

```
user.setPassword(passwordEncoder.encode(registerDto.getPassword()));

        //set the role from role entity
        Set<Role> roles = new HashSet<>();
        Role role = roleRepository.findByName("ROLE_USER").get();
        roles.add(role);
        user.setRole(roles);

        //user is registered in the database
        userRepository.save(user);

        return "User registered successfully";
    }
}
```

Step 3 - update AuthController

```
package com.glo.app.controller;

import com.glo.app.payload.LoginDto;
import com.glo.app.payload.RegisterDto;
import com.glo.app.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private AuthService authService;
    //Constructor based dependency injection
    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    //Spring boot REST api for login
    @PostMapping(value = {"/sign-in","/login"})
    public ResponseEntity<String> login(
            @RequestBody LoginDto loginDto
    )
    {
```

```
        return ResponseEntity.ok(authService.login(loginDto));
    }

    //Spring boot REST api for register
    @PostMapping(value = {"/sign-up","/register"})
    public ResponseEntity<String> register(
            @RequestBody RegisterDto registerDto
    )
    {
        return ResponseEntity.ok(authService.register(registerDto));
    }
}
```

5. References -

Spring Official Document
https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/basic.html